
10.1 Herencia Simple (**extends**) y la Llamada **super**

Dart implementa un modelo de **herencia simple de clase concreta**, lo que significa que una subclase puede derivar directamente de una sola superclase utilizando la palabra clave **extends**.

- La subclase hereda todos los miembros (atributos, getters, setters y métodos) de la superclase, excepto los constructores.
- Para inicializar los atributos de la superclase desde el constructor de la subclase, se utiliza la lista de inicialización con la instrucción **super(...)**.
- Para invocar o extender el comportamiento de un método heredado, se utiliza **super.nombreDelMetodo()**.

```
class Persona {  
  final String nombre;  
  Persona(this.nombre);  
  
  void saludar() => print('Hola, soy $nombre.');}  
  
class Estudiante extends Persona {  
  final String boleta;  
  
  // Inicialización mediante super  
  Estudiante(super.nombre, this.boleta);  
  
  @override  
  void saludar() {  
    super.saludar(); // Reutiliza el saludo base  
    print('Mi boleta es $boleta.');  }  
}
```

10.2 Clases Abstractas (**abstract class**)

Una clase abstracta es una estructura que **no se puede instanciar directamente**. Se utiliza exclusivamente como una plantilla base para definir contratos y comportamiento compartido que deben implementar o extender sus subclases derivables.

- Se declara con la sintaxis **abstract class NombreClase**.
- Puede contener **métodos abstractos** (firmas sin cuerpo ni implementación que obligan a la subclase a implementarlos) y **métodos concretos** (con código reutilizable).

10.3 Interfaces Implícitas (**implements**)

A diferencia de otros lenguajes como Java o C#, **Dart no posee una palabra reservada `interface`

. En su lugar, **cada clase define implícitamente una interfaz** que contiene todos sus miembros públicos.

Cuando una clase utiliza la palabra clave `implements`:

- **Contrato total:** Se compromete a proporcionar la implementación completa de **todos** los atributos, getters, setters y métodos definidos en la clase implementada.
- **Polimorfismo múltiple:** Aunque una clase solo puede heredar (`extends`) de una superclase, puede implementar (`implements`) **múltiples interfaces implícitas** separadas por comas.

```
abstract class Autenticable {
    void login(String usuario, String password);
}

abstract class ExportablePDF {
    void generarPDF();
}

// Implementación múltiple de contratos
class ServicioUsuario implements Autenticable, ExportablePDF {
    @override
    void login(String usuario, String password) { ... }

    @override
    void generarPDF() { ... }
}
```

10.4 Ejemplo

Creen el archivo `bin/d10_herencia_interfaces.dart` en VS Code. El siguiente programa ilustra una arquitectura limpia de repositorios de datos mediante clases abstractas, herencia e interfaces implícitas.

```
/// Módulo D10: Herencia, Clases Abstractas e Interfaces en Dart
/// Demostración de contratos de repositorio, jerarquía de clases y polimorfismo.

void main() {
    print('=====');
    print(' SISTEMA DE ARQUITECTURA DE DATOS (D10 P00) - ESCOM ');
    print('=====');

    // 1. Polimorfismo: Tratamos ambas implementaciones mediante la abstracción del
    // Repositorio
    final RepositorioBase<String> repoMemoria = RepositorioMemoria();
    final RepositorioBase<String> repoCache = RepositorioCacheConNotificación();

    print('--- 1. Operaciones con Repositorio en Memoria ---');
    repoMemoria.guardar('USR-100', 'Carlos Ruiz');
    print('Dato recuperado: ${repoMemoria.obtenerPorId('USR-100')}\n');
```

```

print('--- 2. Operaciones con Repositorio Caché Notificable ---');
repoCache.guardar('USR-202', 'Sofía Ramírez');
print('Dato recuperado: ${repoCache.obtenerPorId('USR-202')}');

// 3. Verificación de tipo e interfaces con el operador 'is'
if (repoCache is Notificable) {
    print('\n--- 3. Invocación de método de Interfaz Implícita ---');
    (repoCache as Notificable).enviarNotificacion('Sincronización de caché
exitosa.');
```

```

    }

    print('\n=====');
```

```

}

/// Contrato genérico base (Clase Abstracta)
abstract class RepositorioBase<T> {
    void guardar(String id, T entidad);
    T? obtenerPorId(String id);

    // Método concreto reutilizable por subclases
    void logOperacion(String mensaje) {
        print(' [LOG REPO] ${DateTime.now().toIso8601String().substring(11, 19)}:
$mensaje');
```

```

    }
}

/// Interfaz implícita para componentes capaces de notificar
abstract class Notificable {
    void enviarNotificacion(String mensaje);
}

/// Subclase concreta que hereda (extends) del RepositorioBase
class RepositorioMemoria extends RepositorioBase<String> {
    final Map<String, String> _tabla = {};

    @override
    void guardar(String id, String entidad) {
        _tabla[id] = entidad;
        logOperacion('Entidad almacenada en memoria RAM con ID: $id');
```

```

    }

    @override
    String? obtenerPorId(String id) {
        logOperacion('Consulta realizada para ID: $id');
        return _tabla[id];
    }
}

/// Subclase que hereda de RepositorioBase e IMPLEMENTA la interfaz Notificable
class RepositorioCacheConNotificación extends RepositorioBase<String> implements
Notificable {
    final Map<String, String> _cache = {};

```

```

@override
void guardar(String id, String entidad) {
  _cache[id] = entidad;
  logOperacion('Dato cacheado localmente con ID: $id');
  enviarNotificacion('Nuevo registro cacheado ($id)');
}

@override
String? obtenerPorId(String id) {
  return _cache[id];
}

// Cumplimiento explícito de la interfaz Notificable
@override
void enviarNotificacion(String mensaje) {
  print(' [NOTIFICACIÓN] Push enviada: "$mensaje"');
}
}

```

Instrucciones de Ejecución en VS Code:

```
dart run bin/d10_herencia_interfaces.dart
```

10.5 Ejercicios Prácticos

Implementen los siguientes dos programas en su entorno local para afianzar el uso de herencia, abstracciones e interfaces implícitas.

Ejercicio 1: Jerarquía de Empleados y Cómputo Salarial Escolar

Crea una estructura de clases para gestionar el personal de la ESCOM:

1. Crea una clase abstracta `EmpleadoEscom` con los atributos `id` (`String`) y `nombre` (`String`). Define el método abstracto `double calcularSueldoMensual()`.
2. Crea una subclase `ProfesorAsignatura` que extienda de `EmpleadoEscom`, agregando los campos `horasImpartidas` (`int`) y `pagoPorHora` (`double`). Implementa el cálculo de sueldo ($horas \times pago$).
3. Crea una subclase `ProfesorTiempoCompleto` que extienda de `EmpleadoEscom`, agregando el campo `sueldoBase` (`double`).
4. En `main()`, crea una lista polimórfica `List<EmpleadoEscom>`, agrega instancias de ambos tipos y recorre la lista imprimiendo el nombre y el sueldo mensual calculado de cada uno.

Ejercicio 2: Motor de Notificaciones Multicanal (Interfaces)

Escribe un sistema de alertas para una app móvil:

1. Define la clase abstracta / interfaz `CanalNotificacion` con el método `void enviarAlerta(String mensaje)`.
 2. Crea tres clases que **implementen** (`implements`) la interfaz `CanalNotificacion`:
 - `NotificacionPush`: Imprime en consola un mensaje simulando un servicio de Firebase Cloud Messaging.
 - `NotificacionEmail`: Imprime en consola la simulación de un correo SMTP.
 - `NotificacionTelegram`: Imprime en consola la simulación del envío de un mensaje a través de un Bot de Telegram.
 3. Crea una función de orden superior `enviarNotificacionMasiva(List<CanalNotificacion> canales, String mensaje)` que ejecute el envío a través de cada canal.
 4. Demuestra su funcionamiento en `main()`.
-